

Real Time Innovations

Tactical Microgrid Example Code Walk-thru

April 9, 2021

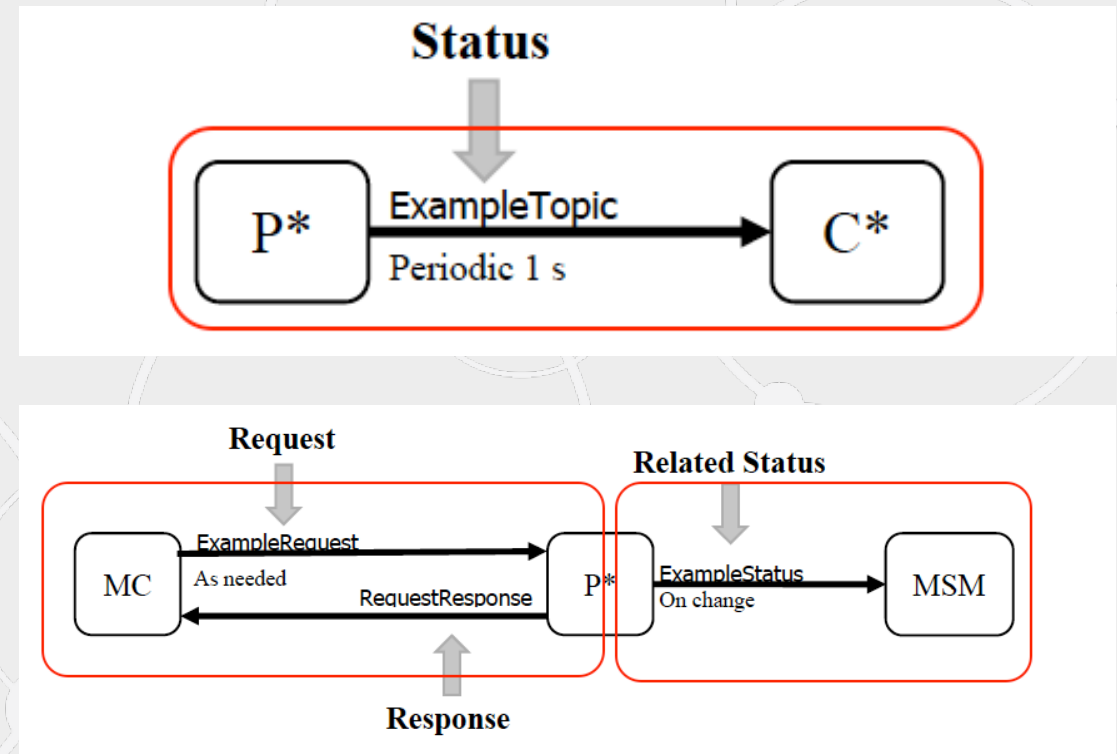
RTI makes the world run better

©2021 Real-Time Innovations, Inc.

Revision 1.5

Software Components

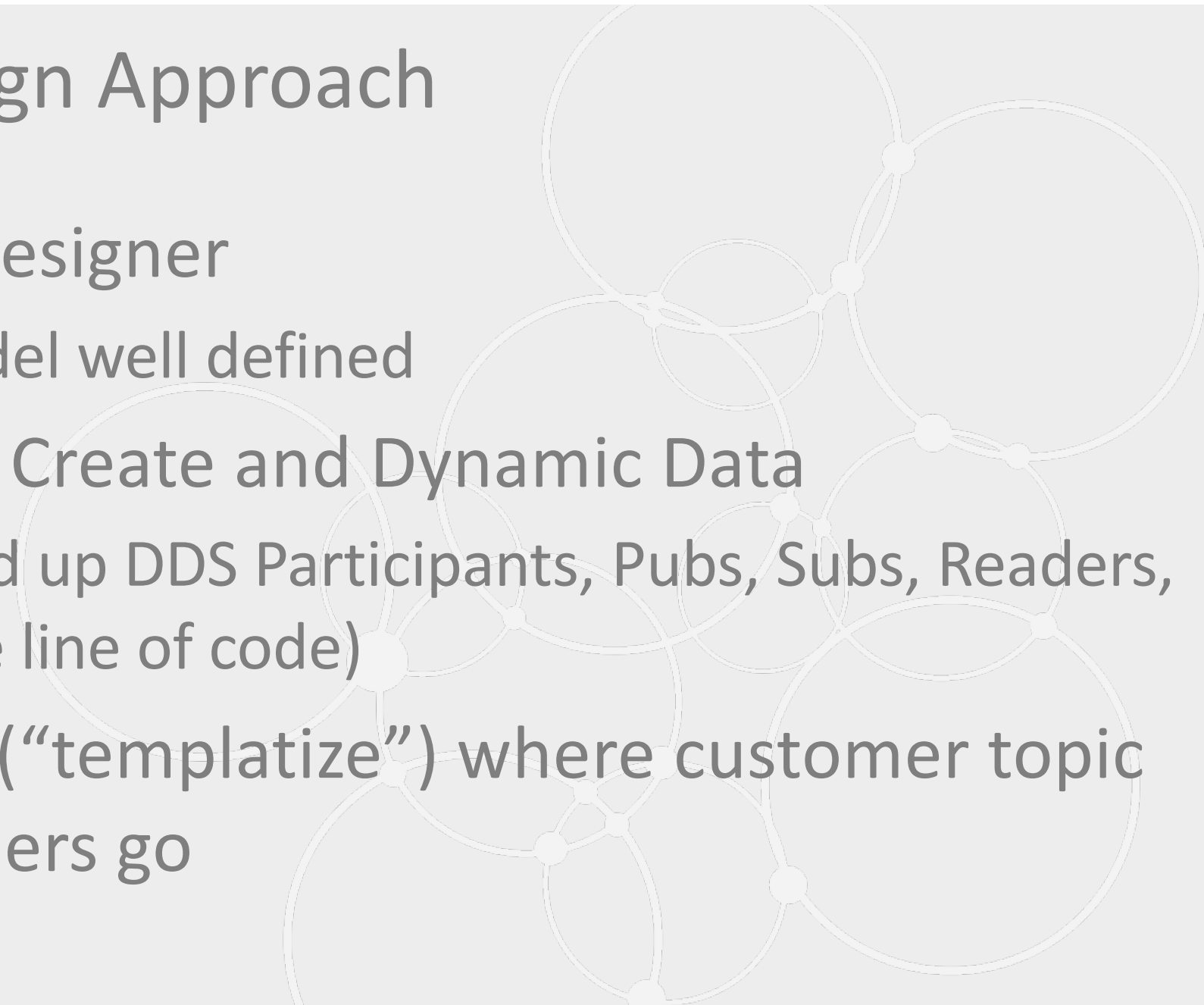
- Tactical Microgrid Spec (TMS)
 - Data Patterns
 - Publish Once (Durable)
 - Periodic
 - Request / Response
 - On-Change
 - Custom (Not implemented)
 - Topics
- Distributed Data Service (DDS)
- Customer Legacy System
 - Gateway Module / Interface between TMS/DDS and Legacy Communications



Issues

- **TMS Spec is complex** in draft form, and hard to get any examples to help confirm understanding
- **Code Complexity** to provide separation of concerns
- **Lumping Complexities:** TMS and application code structure complexity (to isolate legacy adaptation) may be lumped in as DDS complexities
- **DDS Experience** – recommend RTI Virtual Instructor Led Training (24 hour custom training) to get familiar with DDS

Example Design Approach



- Use System Designer
 - Existing model well defined
- Use XML APP Create and Dynamic Data
 - Easy to stand up DDS Participants, Pubs, Subs, Readers, Writers (one line of code)
- Separate out (“templatize”) where customer topic specific handlers go

Implementation



- C++ Language binding
 - MacOS, Visual Studio Code
- Topics are Objects
- Thread based model
 - Thread per topic*
 - separates out separate concerns
 - Note: Threads have little interaction
 - No Locks, Mutex's, synchronization between them
 - Rules to manage Request Correlation to Responses
 - Note clear from spec if one request is outstanding at time, or what one is to do with a 'not OK' Response

Project Directory Structure

- Directories Under Project Directory tms
 - tmsApp (example Application)
 - tmsAppTest-SimMSM
 - MSM simulation used to unit test tmsApp
- .git – git management directory
- .DS_Store (Mac OS file)
- .gitignore (git file to instruct git to ignore built files)

tmsApp – Directory Structure

- Model (tms Data Model Directory) Contains:
 - Tms supplied idl and xml files
 - tms.idl, tms.xml, tms-topics.idl, tms-topics.xml
 - System Designer Project file 'tmsTestApp.rtisdproj'
 - tmsTestApp.xml file
 - The xml file we'll use to to 'standup' all the DDS entities (Participant, Publisher, Subscriber, Readers, Writers and topics)
- cpp – holds source files
- objs – holds the output of the make/build
 - This directory is always removed and recreated with make clean/make (all)
 - It is and its contents are in .gitignor and not checked in the git

tmsApp Directory level files

- makefile – builds tmsApp
- QoS obtained by setting an env var to direct the application to the non-default URL of “Project Directory path/tmsApp/model/tmsTestApp.xml”
- tms.code-workspace and .vscode directory
 - Visual Studio Code workspace and configuration directories

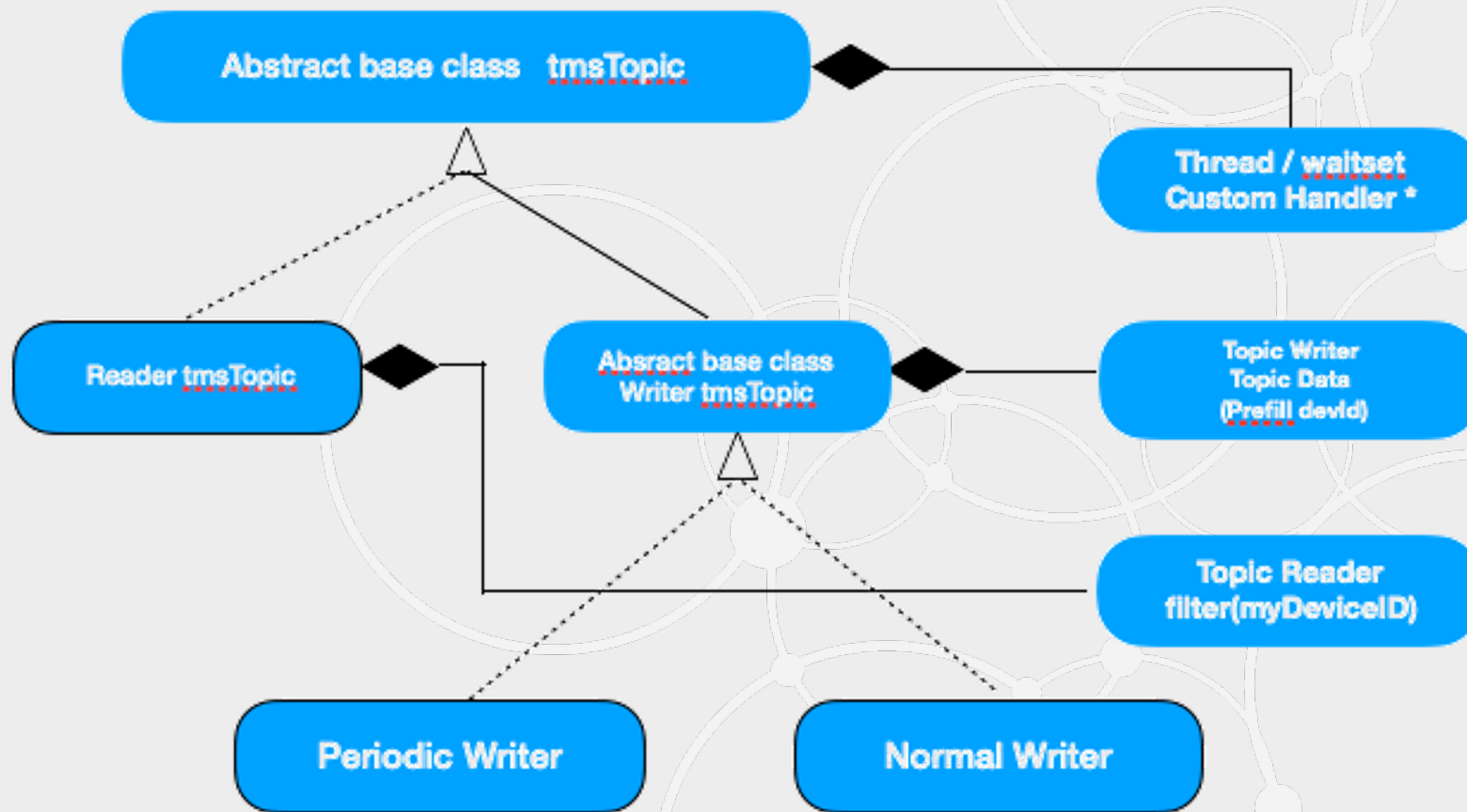
tmsApp cpp files

- tmsTestExample.h
 - Created by rtiddscode gen from the tmsTestApp.xml file and used by the .cxx files to properly reference tms_ specific data structures and names
- tmsCommon.h
 - my common header file for .cxx files during compilation
- tmsTestExampleApp.cxx
 - This is the application C++ top level code

tmsApp cpp files (con't)

- tmsCommPatterns.cxx & .h files
 - Holds the definitions for the tms communications patterns and associated thread control blocks (see tmsCommPatterns.h file). These include
 - ReaderEventThread
 - Required Thread used for all readers
 - WriterEventThread
 - Optional thread to monitor writer topic status
 - PeriodicWriterThread
 - Use to send periodic topics

Topic Objects (1/2)



Topic Objects (2/2)

- Easy user instantiation
 - E.g. `heartbeat = new PeriodicTopic (participant, tms_TOPIC_HEARTBEAT_ENUM, heartbeat_period);`
- Better code factoring
- Topic Objects contain:
 - A thread specific to the communications pattern
 - Local information:
 - Flags to perform common tasks (e.g. prefill DevID field, Filter on DevID “for me”, Echo Responses)
 - Removed the need for global arrays of topics, filters etc.
- Hides the thread-control blocks

tmsCommPatterns – Reader Processing (internal)

- Control Block (ReaderTheadInfo) option
 - echoResponse (true/false)
 - Intended for received requests that require writing a `tms_TOPIC_REQUEST_RESPONSE`
 - Copies the Request SampleId to the RelatedRequestId
 - Sets the Status code to infoBlock `tms_REPLY_code` and `tms_reason` (can be changed in individual topic handlers)

tmsCommPatterns – Writer Processing (internal)

- Optional Pattern Thread
 - Used just to monitor Status Conditions
- You can write anytime using the Writer handle
 - Write Once (e.g., Device Announcement w/Durability)
 - Write “OnChange” of a tms Variable
- User needs to ensure Data to be written is consistent (atomicity)
 - i.e. variables are not changing while you are updating the topic prior to writing

tmsCommPatterns – Periodic Writer

- Writes associated topic periodically
 - As initialized (passed in variable to) by the control block c'tor
- Control Block (PeriodicTheadInfo) option
 - enabled (true/false)
 - Disables the both the call to the handler and the write operation

tmsCommPattern – Topic Handlers

(tmsCommPatternTopicHndlr.cxx & .h)

- Allows customization for topics by user
- Currently only envisioned for all Readers and PeriodicWriters
 - Periodic data is set per specific topic
 - Don't see a regular writer use-case, but easily implemented if needed.

tmsCommPattern – Topic Handlers (Con't)

(tmsCommPatternTopicHndlr.cxx & .h)

- Declare a custom handler in tmsCommPatternTopicHndlr.h
- Write a custom handler in tmsCommPatternTopicHndlr.cxx
- Install the handler in the appropriate handler array location in tmsCommPatternTopicHndlr.cxx
 - reader_handler_ptrs[]
 - Periodic_handler_ptrs[]
- The tmsCommsPattern thread processing does the generic/common processing before and after calling the handler
 - E.g., if on a request receive topic, if EchoResponse enabled, the readerThread will
 - First: Detect data arrival, get the data
 - Second: call the handler (passing a pointer to the info block)
 - Third: If EchoResponse enabled, copy the deviceId and sequenceNumber to the relatedRequestId in requestResponse. Copy the handler set or default status code and status reason to the requestResponse and publish

tmsAppTest-SimMSM – Directory Structure

- Similar to tmsApp Directory Structure but...
 - No model directory
 - Uses the Same model dir as tmsApp
 - Has its own Participant defined in the model
- cpp directory only contains the tmsAppTest-SimMSM.cxx
 - Make file builds and links with the same header files, tmsCommPatters and handler files as in the tmsApp directory
 - tmsCommsPatternTopicHndrs.cxx/.h will hold both tmsApp and tmsAppTest-SimMSM handlers
 - But each instantiates complementary topics and patterns

Mechanisms (1/2)

- Request Sequence Number
 - Manages unique ever incrementing Request Sequence Numbers associated with Device ID in SampleId of requests.
- ReqResQ to track Device Posted Requests with Manager Responses
 - Useful if multiple requests outstanding (not clear if this is a requirement or not)
 - Checks and notifies of overwriting a request that was never responded to
 - This should not happen if command / request / response QoS of reliable reliability is used

Mechanisms (2/2)

- Heartbeat Monitoring tracks all threads
 - User defines a dead-man time out period and hands it into the thread control block — Default = DDS_DURATION_INFINITE
 - Set to non-default causes the associated topic thread_heart_beat_semaphore to be declared “Active”
 - Regardless of dead-man timer each thread topic will set a ‘deadman_fired’ variable in an indexed array of thread_heart_beat_semaphores when ever it runs
 - The Heartbeat handler, if you have one, should check for active thread_heart_beat_semaphore being active and that the dead-man fired
 - In the Device example – this is used to increment a dead-man missed count, and if that reaches 3, the heartbeat is disabled.
 - When ever an active thread_heart_beat_semaphore has the dead-man fire, the count is cleared and the dead-man reset.

Writer/reader Lookup Loops (In Device and MSM Sim Apps)

- Topic readers and writers looked up in loops
 - Defined by user in myWriterIndx & myReaderIndx arrays
- Writer Loop also creates the associated data and, for the device app, fills out the static deviceId field needed for most every writer topic.
- Reader Loop also applies a Content Filter Topic to receive Only Requests and Responses directed to the device

On Change Pattern

- Internal and External Versions of a variable
- If the internal variable state changes, an on-change event is to occur and the associated topic written.
 - The external variable is then set to the internal variable's value so the new topic state on-change is written once per change.
 - Internal versions of the variable can change due to:
 - State of the device changes by user operation (i.e., turn device on)
 - Some condition changes (i.e., fault)
 - Microgrid Manager request command

App and Sim MSM State Machines

- Control device/MSM states
 - Init, PowerUp, SteadyState, Shutdown
- MSM SIM will reset if a new Device Announcement received
- MSM SIM only tracks one device
 - To track multiple devices - would need to add arrays of devices and state machine contexts

Current Status

- <https://github.com/psmass/tms>
- Provides tmsApp and tmsAppTest-SimMSM
 - MSM Sim test code only implemented enough to keep app stimulated
 - E.g., does not even look at device discovery topics
- Eight planned topics working
 - Between the App and Test App – all planned patterns working